

YZM304 — Derin Öğrenme · Proje 4

Mehmet Ali Topkara · 23291093

## Sunum Konuşma Metni — Proje 4

**Hedef süre:** ~10–12 dk · **Slayt sayısı:** 16 · **Demo süresi:** ~2–3 dk

Her slayt için 30–50 saniyelik konuşma metni. Akıcı bir tonda okunduğunda toplam ~9–10 dk + canlı demo 2–3 dk = ~12 dk olur. Anlatırken metni ezbere okumak yerine ana cümleleri tutup kendi sözlerine çevirebilirsin.

### Slayt 1 — Başlık (≈30 sn)

Merhaba arkadaşlar. Bugün size YZM304 dersinin 4. proje çalışmamı sunacağım: "**Oyun Ekran Görüntülerinden Oyun Tespiti**". Çalışmamda klasik yapay sinir ağı (MLP), sıfırdan eğitilen bir CNN ve **transfer learning** tabanlı üç modern mimariyi (ResNet50, EfficientNetB0 ve Vision Transformer) **aynı görev üzerinde** karşılaştırdım. Buna ek olarak modelleri lokal bir web demosu üzerinden interaktif test edilebilir hale getirdim. Sunumun sonunda canlı demoyu da göstereceğim.

### Slayt 2 — Neden Bu Konu? (≈40 sn)

Konuyu seçerken iki şey beni yönlendirdi. Birincisi **pratik ihtiyaç**: oyun pazarı 2024 itibarıyla yaklaşık 200 milyar dolara ulaştı, Twitch ve YouTube Gaming gibi platformlara her gün milyonlarca saatlik gameplay görüntüsü yükleniyor. Bu hacimde manuel etiketleme imkansız hale geldi; arama, içerik moderasyonu, telif kontrolü ve öneri sistemleri için **otomatik sınıflandırma** kritik bir altyapı haline gelmiş durumda. İkincisi ise **çeşitlilik**. Sınıf arkadaşlarımdan büyük çoğunluğu sağlık ve insan tabanlı görüntü sınıflandırma problemlerini — tıbbi görüntüler, yüz tanıma, duygu analizi gibi — tercih etti. Ben özellikle farklı bir bağlam denemek istedim ve oyun ekran görüntülerini seçtim. Hem sektörel olarak güncel, hem de görsel zorluk açısından zengin bir problem alanı sunuyor.

## Slayt 3 — Problem, Araştırma Sorusu, Birincil Metrik (≈45 sn)

**Problemim** şu: tek bir oyun ekran görüntüsü verildiğinde, modelin görseli 10 önceden tanımlı sınıftan birine atfetmesi. Buna **closed-set sınıflandırma** denir; model her zaman bir tahmin vermek zorundadır.

**Araştırma sorum** üç başlıklı: görüntü yapısını yok sayan MLP, sıfırdan eğitilen klasik CNN ve transfer learning modelleri aynı veri setinde nasıl performans gösterir? Mimari değişiminin ve **pretrained ağırlıkların** etkisi sayısal olarak ne kadardır?

**Hipotezim** şuydu: MLP düşük performans verir; CNN scratch belirgin bir sıçrama yapar; transfer learning marjinal bir iyileştirme sağlar — fakat maliyet açısından çok daha pahalı olur.

**Birincil metrik olarak Macro-F1** seçtim. Veri dengeli olduğu için accuracy ve weighted-F1 yakın çıkıyor; ancak Macro-F1 her sınıfa eşit ağırlık veriyor ve precision–recall'u harmonik ortalama ile birleştiriyor. Bu, multi-class dengeli sınıflandırma için akademik standarttır.

## Slayt 4 — Veri Seti (≈35 sn)

Çalışmada **Kaggle** platformunda yayınlanan **Gameplay Images** veri setini kullandım. Veri seti 10 farklı popüler oyundan toplam 10.000 ekran görüntüsü içeriyor; her sınıf için 1000 örnek var ve dağılım sağdaki grafikte de görüldüğü gibi mükemmel şekilde dengeli. Görseller 640×360 piksel çözünürlükte PNG formatında.

Sınıflar şöyle: Among Us, Apex Legends, Fortnite, Forza Horizon, Free Fire, Genshin Impact, God of War, Minecraft, Roblox ve Terraria. Hem kazi/sandbox hem MMORPG hem FPS gibi farklı türler bilinçli olarak seçildi — model çeşitlilikle test edilsin diye.

## Slayt 5 — Veri Ön İşleme ve Bölme (≈40 sn)

Ham veride iki temel ön işleme adımı uyguladım. Birincisi **format dönüşümü**: ham görsellerin yaklaşık %20'si RGBA, yani alfa kanallı formatta; eğitim öncesinde tümünü RGB'ye çevirdim. İkincisi **boyut normalize**: tüm görselleri 224×224 piksele resize ettim, böylece ImageNet üzerinde önceden eğitilmiş modellerin beklediği girdi boyutuyla uyumlu hale geldiler.

Eğitim setinde **augmentation** uyguladım: rastgele kırpma, yatay çevirme, renk titreşimi ve  $\pm 10^\circ$  rotation. Validation ve test setlerinde sadece deterministik resize + center crop kullandım. ImageNet ortalama–standart sapma değerleriyle normalize ettim.

Veri setini **stratified split** ile %70 eğitim, %15 doğrulama, %15 test oranlarında böldüm — yani 7000 train, 1500 val, 1500 test. Stratified yöntemi her sınıftan eşit oranda örnek alıyor; reproducibility için seed'i 42'de sabitledim. Test seti hiçbir aşamada eğitime karışmadı.

## Slayt 6 — MLP Mimarisi ve Eğitimi (≈50 sn)

Solda gördüğünüz şema **MLP baseline** mimarisi. Yapay sinir ağının klasik halidir: görüntüyü düz bir vektöre çeviriyor — 224×224×3 yani **150.528 piksel değeri** — ve üç tam bağlı katmandan geçiriyor. Önce 256 nöronluk gizli katman, sonra 128 nöronluk ikinci gizli katman, son olarak 10 sınıflık çıktı katmanı. Her gizli katmandan sonra ReLU aktivasyonu ve Dropout 0.3 var.

Toplam parametre sayısı **38,57 milyon** ve hepsi tam bağlı katmanlarda. Bu mimari **görüntüdeki uzamsal yapıyı tamamen yok sayar** — komşuluk, kenar, doku gibi bilgileri kullanmıyor; bu yüzden pedagojik baseline olarak işe yarıyor.

Eğitim parametreleri: AdamW optimizer ( $lr=1e-3$ ), CosineAnnealingLR scheduler, batch size 64, **20 epoch maksimum, early stopping patience=5**. Eğitim 11,8 dakika sürdü. **19. epoch'ta best val\_acc 0,5053'e** ulaştık. 20. epoch'ta no improve 1/5 ile durdu. Yani MLP rastgele tahminin (%10) üstüne çıkıyor ama uzamsal yapıyı kullanamadığı için %50'de takılıp kalıyor.

## Slayt 7 — CNN Scratch Mimarisi ve Eğitimi (≈50 sn)

Üstte gördüğünüz şema sıfırdan eğitilen CNN baseline'im. Klasik **VGG-mini** tarzında dört evrişim bloğundan oluşuyor. Her blokta bir 3×3 convolution, batch normalization, ReLU aktivasyonu ve 2×2 maxpooling var. Filter sayıları her blokta ikiye katlanıyor — 32, 64, 128, 256 — ve feature map çözünürlüğü her blokta yarıya düşüyor: 224 → 112 → 56 → 28 → 14.

Son convolution bloğunun çıktısı global average pooling ile 256 boyutlu bir vektöre indirgeniyor; ardından 128 birimlik bir tam bağlı katmandan geçiyor ve 10 sınıflık tahmin üretiliyor. **Toplam parametre sayısı sadece 0,42 milyon — yani MLP'den 92 kat daha az.** Ama uzamsal yapıyı sömürdüğü için sonuçları çok daha iyi.

Aynı eğitim protokolünü uyguladım: AdamW (lr=1e-3), CosineAnnealingLR, AMP mixed precision, **20 epoch + early stopping patience=5**. Eğitim 11,1 dakika sürdü. **19. epoch'ta best val\_acc 0,9680'e** ulaştık. Yani ImageNet pretrained kullanmadan, sıfırdan, sadece 0,42 milyon parametre ve 11 dakikada %96 üzeri val accuracy.

## Slayt 8 — Transfer Learning + 3 Modern Mimari (≈45 sn)

Şimdi transfer learning ailesine geçelim. Kavram basit: ImageNet üzerinde önceden eğitilmiş ağırlıkları al, son sınıflandırma katmanını kendi probleme göre değiştir, tüm ağırlıkları kendi verinde ince ayarla. Tüm katmanları fine-tune ettim, backbone'u dondurmadım.

Üç farklı paradigma temsilcisi seçtim. **ResNet50** klasik CNN'in 2015 sembolü, residual bağlantılarla derin ağ eğitimini mümkün kılar; 23,5 milyon parametreyle 10,6 dakikada %99,13 macro-F1'e ulaştı.

**EfficientNetB0** modern verimli CNN — MBConv blokları ve compound scaling fikri var. Sadece 4 milyon parametre ve 9,5 dakika eğitimle %99,07 macro-F1; aynı doğruluk ResNet'ten 5–6 kat daha az parametreyle.

**ViT-Base/16** transformer mimarisini görüntüye taşır — patch tokenization. 86 milyon parametre, 19,5 dakika eğitim, %99,07 macro-F1. Üçü de timm kütüphanesi üzerinden tutarlı API ile yüklendi. Tüm modeller 20 epoch + early stopping patience=5 ile aynı protokolde eğitildi.

## Slayt 9 — Eğitim Protokolü ve Alternatifler (≈40 sn)

Adil karşılaştırma için **5 modeli aynı protokolde** eğittim. Optimizer AdamW, scheduler CosineAnnealingLR, loss CrossEntropyLoss. Batch size baseline'larda 32–64, ViT'te VRAM nedeniyle 16. Maksimum 20 epoch, early stopping patience 5. AMP mixed precision sayesinde transfer ve CNN scratch eğitimleri yaklaşık %40 hızlandı, doğruluğa zarar vermedi. Reproducibility için seed=42, deterministic dataloader.

Sağdaki tabloda **düşünüp uygulamadığım alternatifler** var. Sıfırdan eğitsem 10K görsel yetmezdi, doğruluk %20–30 düşerdi. Frozen backbone hızlı ama domain shift yüzünden %3–5 doğruluk kaybı. LoRA dev modeller için anlamlı, 86M ViT için marjinal. Mixup/CutMix %1–2 ek doğruluk verebilir ama mimari karşılaştırması odağını dağıtırdı. K-Fold CV ise 5 kat hesaplama maliyeti getirirdi; tek tahmin için stratified split yeterli olduğundan tercih etmedim.

## Slayt 10 — Sonuçlar: 5 Model Test Performansı (≈55 sn)

Test seti üzerindeki 5 modelin performans tablosu. **Macro-F1 kolonu birincil metrik** olduğu için yeşil arka planla vurguladım.

En çarpıcı bulgu: **MLP ile transfer learning arası yaklaşık 0,51 macro-F1 farkı var**. Yani görüntü yapısını sömüren mimarilerin değeri çok net görülüyor. MLP %47,94'te kalırken CNN scratch tek atlamada %97,33'e çıkıyor.

**CNN scratch'in performansı şaşırtıcı**: yalnızca 0,42 milyon parametre ile 0,9733 macro-F1'e ulaşıyor. ResNet50'nin 0,9913'üyle arasında sadece 0,018 fark var — yani transfer learning'in marjinal katkısı bu görevde küçük; **esas sıçrama MLP→CNN geçişinde yaşanıyor**.

Transfer learning grubunda fark istatistiksel gürültü içinde — sadece %0,06. EfficientNetB0 21 kat daha az parametre, 6 kat daha küçük dosyayla pratik açıdan en akıllı tercih. ViT en pahalı ama doğruluk avantajı yok.

## Slayt 11 — Eğitim Eğrileri Karşılaştırması (≈35 sn)

Bu grafikte 5 modelin eğitim eğrilerini üst üste koydum. Sol panellerde loss, sağ panellerde accuracy görüyorsunuz; train ve val ayrı.

**MLP** gri çizgi, en altta plato yapıyor — %50 civarında takılıyor, daha fazla epoch ona yardım etmiyor.

**CNN Scratch** turuncu, daha yavaş ama düzenli yükseliş. **Transfer learning modelleri** ilk birkaç epoch'tan sonra zaten %95 üstüne çıkıyor — bu, ImageNet ağırlıklarının gücünü doğrudan gösteriyor. Modelin görsel özellik çıkarma yeteneği hazır geldiği için adapte olması çok hızlı.

## Slayt 12 — Sınıf-Bazlı Macro-F1 Analizi (≈45 sn)

Üstteki yatay bar 5 modelin macro-F1'ini özetliyor; net hiyerarşi: MLP %48, CNN scratch %97, transfer modelleri %99.

Altteki grafikte ise her sınıf için 5 modelin F1 değerlerini grouped bar chart olarak gösteriyorum. Burada birkaç ilginç bulgu var: MLP'nin Apex Legends ve Free Fire F1'i 0,2 civarında — yani FPS oyunlarını birbirine karıştırıyor. Çünkü uzamsal yapıyı kullanmadığı için renk dağılımına bakıyor, bu iki oyun da benzer renk paletine sahip.

CNN scratch ve transfer modelleri tüm sınıflarda 0,95 üstü performans sağlıyor. En zor sınıf yine Apex Legends; benzer HUD elementleri taşıyan diğer FPS'lerle karıştırılma riski tüm modellerde mevcut. Minecraft her modelde mükemmel — bloklu pikseller eşsiz bir görsel imza.

## Slayt 13 — Hata Analizi (Confusion Matrices) (≈35 sn)

5 modelin normalize confusion matrix'ini yan yana koydum. Diagonal recall'a karşılık geliyor; koyu kırmızı doğru tahmin demek.

**MLP'nin matrisi dağınık** — özellikle Apex Legends, Free Fire ve God of War satırlarında yanlış tahminler yoğun. **CNN Scratch ve transfer** modellerinin diagonal'leri belirgin şekilde temizlenir; yanlışlar matriste seyrek noktalar halinde kalıyor.

Tüm modellerde en zor karışan sınıf çifti **Apex Legends ↔ Free Fire** — her ikisi de battle royale FPS, benzer HUD elementleri (mini harita, sağlık barı, silah göstergesi) taşıdığı için karıştırılması beklenen bir durum.

## Slayt 14 — Grad-CAM ve EigenCAM (≈45 sn)

Bir model %99 doğruluk vermesi yetmez; **kararına hangi piksellerin etki ettiği** sorusu da önemlidir. Heatmap görselleştirmesi bu görünmez mantığı açığa çıkarır — model debug ve kullanıcı güveni açısından kritik.

CNN'lerde **GradCAM** kullandım: son convolution katmandaki gradient'i activation ile çarpıyor, sınıf-özel bir harita üretiyor. CNN scratch için son conv bloğu, ResNet için layer4'ün son bloğu, EfficientNet için son MBConv stage'i.

Ama ViT'te bir sorun çıktı: ViT softmax saturation yapıyor, confidence 1.0'a ulaşıyor ve vanilla GradCAM'in gradient'i tam sıfır oluyor. Bu yüzden ViT için **EigenCAM** kullandım: gradient gerektirmiyor, son block aktivasyonlarının SVD'sini alıyor — gradient-free, robust.

MLP için heatmap üretemiyoruz; tam bağlı katmanların uzamsal feature map'i yok. Pratik anekdot: Demoda bir Roblox görselinde ViT yanlış cevap verdi; EigenCAM modelin avatarlar yerine gökyüzüne baktığını gösterdi. Hata sebebi anında anlaşıldı.

## Slayt 15 — Canlı Demo + Belirsizlik Göstergesi (≈30 sn setup + 2–3 dk demo)

### Setup konuşması (slayt görüldüğünde):

Şimdi canlı demoya geçiyorum. Lokal bir web uygulaması yazdım — backend FastAPI, frontend Vite + React + TypeScript + Tailwind. Üç adımdan oluşuyor: görsel yükleme, model seçimi (5 modelden tek bir tane ya da "tümünü karşılaştır") ve sonuç görüntüleme.

En kritik özelliği **belirsizlik göstergesi**: Shannon entropy ve top-1/top-2 margin'den 3 seviyeli rozet üretiyor — yeşil "kesin", sarı "şüpheli", kırmızı "belirsiz". Comparison modunda 3+ model "low" işaretlerse, görsel muhtemelen 10 sınıf dışı, yani out-of-distribution sinyali.

### Canlı demo akışı (sırayla göster):

- 1. Konsensüs örneği** — Minecraft sample'ını seç → Tümünü Karşılaştır → 5 model de doğru, banner yeşil. *"Burada 5 model de aynı cevabı veriyor, hepsi yüksek güven."*
- 2. Çelişki örneği** — Fortnite sample'ı → Tümünü Karşılaştır → bazı modellerin yanıldığı vaka (özellikle MLP'nin yanılması ve transfer modellerin doğru bulması). *"Bakın, MLP burada Among Us diyor — uzamsal yapıyı kullanamadığı için görseli yanlış yorumladı; CNN scratch doğru, transfer modeller de doğru."*

3. **Heatmap göster** — bir modelin heatmap'ini aç, ne'ye baktığını göster.

4. **OOD örneği (varsa)** — listede olmayan bir oyun (Cyberpunk vb.) yükle → çoğu model "belirsiz" → kırmızı OOD banner. *"Listede olmayan bir oyun yüklediğimde model tahmin etmek zorunda ama belirsizlik göstergesi 'bilmiyorum' sinyalini veriyor."*

## Slayt 16 — Sonuç Değerlendirmesi, Sınırlamalar, Erişim (≈55 sn)

Toparlayalım. Üç ana çıkarım var:

Birincisi, **mimari fark çoğu zaman maliyet farkıdır**. Bu görevde 0,42 milyon parametrelili CNN scratch ile 86 milyon parametrelili ViT arasında macro-F1 farkı sadece 0,018. Doğruluk değil, verimlilik kararı önemli. Production'a koyacaksanız EfficientNetB0 en akıllı tercih.

İkincisi, **transfer learning'in en büyük katkısı erken epoch'larda görülür** — eğitim hızı ve veri ihtiyacı açısından. Yeterli veri ve zaman varsa sıfırdan eğitilen CNN bile yüksek doğruluğa ulaşır.

Üçüncüsü, **confidence ≠ correctness** — model %87 emin olabilir ve yine de yanılabilir. Belirsizlik göstergesi bunu kullanıcıya anlık olarak gösteriyor.

Sınırlamalar açısından gelecek çalışma için: gerçek OOD detection (Mahalanobis, energy-based) eklenebilir, dataset daha çeşitli sahneleri kapsayacak şekilde genişletilebilir, confidence calibration yapılabilir, multi-label genişletme düşünülebilir, mobil deploy için EfficientNet ONNX'e çevrilebilir.

**Blog yazısı GitHub'da:** alttaki şemada yol var — github.com'dan MALiTopkara hesabına, oradan YZM304-Derin-Ogrenme reposuna, içindeki proje4 klasörüne, oradan blog.md dosyasına. Beni dinlediğiniz için teşekkür ederim. Sorularınızı bekliyorum.

## Sıkça Sorulabilecek Sorular ve Hazır Cevaplar

Aşağıdaki cevaplar 20–30 saniyede verilebilecek özlü cevaplar. Soruyu önce kısa onayla ("güzel soru / iyi noktaya değindiniz"), sonra cevabı ver.

### Metodoloji

**S1: Pretrained kullanmasaydınız ne olurdu?**



CNN scratch sonucum bu sorunun zaten cevabı — sıfırdan eğitilen 0,42M parametrelili CNN, 11 dakikada %96,8 val accuracy aldı. Ama bu küçük dataset'e özgü — 10K görselden çok daha az veriyle çalışsam transfer learning'in avantajı çok daha belirgin olurdu.

### S2: Cross-validation neden yapmadınız?

5-fold CV beş kat hesaplama maliyeti getirirdi ve tek bir nihai tahmin için stratified train/val/test split yeterli veri ayırımını sağlıyor. Asıl odağım mimari karşılaştırması, hiperparametre tuning değildi. Future work olarak listelenebilir.

### S3: Hiperparametre tuning nasıl yapıldı?

Hiperparametreleri tune etmedim — kasıtlı olarak. Amacım modelleri aynı protokolde adil karşılaştırmaktı. AdamW + CosineLR +  $1e-4$  lr (transfer için) ve  $1e-3$  lr (baseline için) standart başlangıç noktaları.

### S4: Augmentation sade — Mixup/CutMix neden yok?

Mixup veya CutMix %1–2 ek doğruluk verebilirdi ama bu görevde transfer modeller zaten %99'da. Daha agresif augmentation, mimari karşılaştırma sinyalini bulanıklaştırırdı. Future work.

### S5: AMP doğruluğu etkiledi mi?

Hayır. AMP'siz baseline ile AMP'li versiyonun ResNet için val accuracy'lerini karşılaştırdım — fark %0,0 mertebesinde, gürültü içinde. AMP ana etki olarak %40 hızlanma sağladı.

## İstatistik / Sonuçlar

### S6: %99 başarı çok yüksek — overfit veya data leakage olabilir mi?

Bunu kontrol ettim. Test set tamamen ayrı tutuldu; sadece evaluate.py'de kullanıldı. Stratified split seed sabit, deterministic. Eğitim sırasında test seti hiç görülmedi. Confusion matrix'ler de farklı sınıflarda farklı performans gösteriyor — leakage olsa hepsi mükemmel olurdu. Apex Legends'in daha düşük recall'u, gerçek bir öğrenme sinyalidir.

### S7: 1500 örnekte %0,06 macro-F1 farkı anlamlı mı?

Hayır. Wilson confidence interval ile %95 güvenle tek bir modelin doğruluk aralığı yaklaşık  $\pm 0,5$ . Modeller arası 0,06 fark bu aralığın çok altında — gürültü. Asıl ayırım maliyet kolonlarında.

#### S8: ViT en yüksek validation, test'te eşit — neden?

ViT validasyonda 0,9960, testte 0,9907 — yaklaşık %0,5 düşüş. Küçük overfit göstergesi. ViT'in 86M parametresi bu dataset için kapasite fazlası; daha çok veri ister. Inductive bias'ı az olduğu için val/test ayrışmasında daha kırılğan.

#### S9: Birincil metrik neden Macro-F1?

Veri dengeli olduğu için accuracy ve weighted-F1 yakın çıkıyor; Macro-F1 her sınıfa eşit ağırlık veriyor ve precision–recall'u dengeli birleştiriyor. Multi-class balanced classification için akademik standart. Ayrıca per-class breakdown ile uyumlu — Apex Legends'ın F1'i düşük dediğimde doğrudan hangi sınıfın zayıf olduğunu görebiliyoruz.

#### S10: Confidence calibration yaptınız mı?

Hayır, calibration projenin scope'u dışındaydı. Demoda gördüğünüz gibi, model %87 güvenle yanılabilir — bu calibration eksikliğinin göstergesi. Future work'te temperature scaling veya focal loss denenebilir.

## Mimari

#### S11: MLP'nin parametre sayısı niye CNN'den çok fazla?

Çünkü MLP girdiyi tamamen düzleştiriyor:  $224 \times 224 \times 3 = 150.528$  piksel değerini direkt 256 birimlik tam bağlı katmana bağlıyor; bu tek katmanda 38 milyon parametre var. CNN ise  $3 \times 3$  filtreleri tüm görsel boyunca paylaşıyor — convolution'ın doğal parametre verimliliği. Bu zaten görüntü problemlerinde CNN'in MLP'yi neden ezdiğinin kanıtı.

#### S12: ViT neden EigenCAM, CNN'ler GradCAM?

ViT softmax saturation yapıyor — top-1 confidence tam 1,0, diğer sınıflar  $1e-7$  mertebesinde. Vanilla GradCAM softmax çıktıyı geri yayıp aktivasyonların gradientini alıyor; saturation'da gradient tam sıfır → heatmap tamamen siyah. EigenCAM gradient kullanmıyor, son block aktivasyonlarının SVD'sini alıyor.

**S13: ViT 86M parametre, EfficientNet 4M — niye eşit doğruluk?**

Bu dataset için 4M parametre yeterli kapasite. ViT'in 86M parametresi daha karmaşık görevlerde (ImageNet, COCO) avantaj sağlar; 10 sınıflık nispeten basit bir dataset için kapasite fazlası. Inductive bias eksikliği de küçük dataset'te dezavantaj.

**S14: CNN scratch %97'ye çıktı — transfer learning gerçekten gerekli mi?**

Bu görev için marjinal — sadece 0,018 fark. Ama varsayımlar değişirse: daha az veriniz olsa, daha az zamanınız olsa, ya da mimariyi henüz seçemediyseniz; transfer learning hâlâ büyük avantaj. Pratik tavsiyem: önce küçük bir model (CNN scratch) deneyin, yetmezse transfer'e geçin.

**Genelleme****S15: Listedeki 10 dışında oyun gelirse?**

Closed-set softmax bir tahmin vermek zorunda — görsel olarak en yakın sınıfı seçer. Demoda gördünüz: 3 modelin tahminleri farklı çıkar. Belirsizlik göstergesi bu durumu yakalamak için var: 3+ model "low" confidence verirse banner kırmızı OOD uyarısı çıkarıyor.

**S16: Yeni bir oyun çıkarsa modeli yeniden mi eğiteceksiniz?**

Evet, full retraining gerekir — bu closed-set'in doğal sınırı. Pratikte open-set yöntemler (Mahalanobis, energy-based scoring, rejection learning) bu problemi azaltır ama tamamen çözmez.

**S17: Gerçek dünya görselleri (low-res, edit'li) nasıl davranır?**

Test setim Kaggle dataset'inden — 640×360 sabit boyut, gameplay sahneleri. Gerçek dünyada compress edilmiş, fotoğrafı çekilmiş, overlay eklenmiş görseller distribution shift yaratır; performans muhtemelen düşer. Production sistemde monitoring gerekir.

**Mühendislik****S18: Streamlit/Gradio değil, neden React?**

Streamlit/Gradio çok hızlı prototip için harika ama yan yana karşılaştırma görünümü, animasyonlu güven barları ve heatmap toggle gibi özellikler için kontrolü kısıtlı. React + Tailwind profesyonel görünüm + tam kontrol veriyor. Trade-off: scaffold süresi biraz daha uzun.

### S19: Production'a nasıl götürürsünüz?

Birkaç adım: model'i ONNX'e çevir (mobil ve cross-platform), Docker container'a sar, CDN arkasına koy. Versioning için MLflow veya DVC. Monitoring: prediction latency, confidence dağılımı drift'i, OOD oranı. A/B test için trafik split.

### S20: Belirsizlik göstergesinin eşikleri nasıl seçildi?

Heuristic:  $\max \text{prob} \geq 0,85 + \text{margin} \geq 0,50 + \text{entropy} \leq 0,20 \rightarrow \text{"high"}$ . Test set dağılımına bakarak belirledim. Production'da validation set üzerinde calibrate edilmeli — bu projenin demo amacı için yeterli ama formal calibration eksik.

## Teknik Detay

### S21: Eğitim sırasında crash olursa?

Aslında EfficientNet eğitiminde CUDA illegal memory access olduğu için train.py'a `--resume` desteği ekledim: her epoch sonu checkpoint kaydeder (model + optimizer + scheduler + history), `--resume` ile kaldığı yerden devam eder. AMP + incremental history.csv yazımı + empty\_cache çağrıları crash riskini azalttı.

### S22: Inference time neden 9–12 ms aralığında?

Batch size 32, RTX 5070 Ti GPU, fp32. Tek görsel ölçümünde daha yavaş olur (warmup + tek-örnek overhead). Test seti 47 batch, toplam süre / örnek sayısı = ortalama 9–12 ms.

### S23: Frontend'in OOD tespitinde yanlış pozitif olur mu?

Evet, eşikler heuristic. Mesela bir oyunun bilmediği yeni bir mod görseli "OOD" olarak işaretlenebilir. Demoda zor in-distribution örneklerde bazen 2 model şüpheli ama tahmin yine doğru çıkıyor. Calibration ile iyileştirilebilir.

## Sunum Sırasında İpuçları

1. **Tempo:** ortalama 35–45 sn/slayt hedefte tut. Slayt 6–7 (MLP/CNN diyagramları), 10 (sonuç tablosu), 16 (kapanış) biraz daha uzun konuşulabilir; başlık ve referanslar hızlı geç.
2. **Demo paniği yapma:** Backend ve frontend zaten başlatılmış olmalı sunum öncesi. `localhost:5173` tarayıcıda açık dur. Ctrl+Shift+R ile cache temizle.
3. **Hikaye akışı:** MLP→CNN→Transfer akışını 3 adımda anlat — "uzamsal yapıyı yok sayan model %50, sömüren model %97, pretrained model %99". Bu tek cümle sunumun özeti.
4. **F1 vurgusu:** "macro-F1 birincil metriğimiz" cümlesini en az 3 farklı slaytta tekrarla — soru gelirse hazır cevap olsun.
5. **Çeşitlilik anekdotu:** Slayt 2'de "sınıf arkadaşlarım sağlık üzerine, ben oyun seçtim" — bu kişisel bir not, samimi bir ton kat.
6. **Bilmiyorsan:** "Bu yönü detaylı incelemedim, future work olarak not aldım" demek tamamen kabul edilebilir.

---

GitHub: [github.com/MAlitopkara/YZM304-Derin-Ogrenme/tree/main/proje4](https://github.com/MAlitopkara/YZM304-Derin-Ogrenme/tree/main/proje4)